BIG DATA · Academic year 2026-2027

Assignment 2

Optimised Dense and Sparse Matrix Multiplication in C

Grado en Ciencia e Ingeniería de Datos
Universidad de Las Palmas de Gran Canaria

Carlos Falcón Brito

Repository: https://github.com/Cafabri/Assigment2

Abstract

The triple-loop `ijk` multiplication of Assignment 1 was compared in C with three optimised dense kernels (loop reordering `ikj`, cache blocking and a transposed operand) and with a CSR sparse multiplication (Gustavson's algorithm), on the performance cores of an Intel Core i7-13700KF (Windows 11), under a budget of 15 minutes per run. Every kernel adds the products of each element in the same order, and all results were bit-identical to `ijk` and agreed with NumPy and SciPy. For dense matrices, `ikj` needed 1.6–8.6 times less time than `ijk` for $500 \leq n \leq 4000$ and the blocked kernel 1.7–16.4 times less; the blocked kernel kept a constant cost of 0.22 ns per multiply-add from $n = 500$ to 4000, without the jump at $n = 2000$ seen in Assignment 1. Compiler vectorisation (AVX2) made the blocked kernel a further 2.5–2.8 times faster, up to 45.8 times faster than `ijk` at $n = 4000$, and changed the time of the kernel it could not vectorise by at most 4% from $n = 200$. At $n = 2000$, the CSR multiplication was faster than the scalar blocked kernel up to about 55% density for uniformly placed nonzeros and 57% for band matrices, and faster than the vectorised one up to 33% and 35%. The CSR matrices needed less storage than the dense arrays up to 50% (uniform) and 56% (band) density, even when the product was almost full. CSR completed every tested size up to $n = 32\,000$, where uniform inputs with 1% density gave a product with 96% nonzeros and 11.2 GiB of storage. These results describe these programs on this computer.

1 Introduction and objective

1.1 Motivation

In Assignment 1 the `ijk` multiplication in C followed the n^3 model only up to $n = 1500$: between $n = 1500$ and 2000 its cost per multiply-add rose from 0.55 to 1.30 ns, which was attributed, without a direct measurement, to the column-by-column reading of B once B no longer fits in the L3 cache or in the reach of the TLB. If that explanation is right, changing only the order in which memory is read should remove most of the jump. A second way to save time is not to do the work at all: a sparse format stores and multiplies only the nonzeros, but the indexing it needs and the fill-in of the product can cancel that advantage [5].

1.2 Research questions

> *(A) How do loop reordering, cache blocking and a transposed operand change the runtime and memory of the* ***`ijk`*** *dense multiplication of Assignment 1, and how much of the change comes from compiler vectorisation?*
> *(B) Under which tested densities and nonzero distributions does a CSR sparse multiplication need less time or memory than a dense method on the same logical matrices?*

1.3 Variables

Factors varied: the dense method, the matrix size n, the block size b and the build (scalar or AVX2) in Part A; the density d of the inputs (the percentage of nonzero entries; the sparsity is $100\% - d$), the distribution of the nonzeros (uniform or band), n and the method in Part B. **Outcomes:** kernel time, the separately timed conversion and transposition costs, the nonzeros of the product and the memory. **Kept fixed:** language (C), compiler and flags within each build, float64, input values, machine, single thread.

1.4 Hypotheses

Table 1 lists the predictions, derived from Section 2; those on the transposed operand and on the sparse crossover were formulated after a pilot run (Section 3.4), whose timings may have influenced them.

2 Background

2.1 Dense optimisations and vectorisation

The triple loop performs n^3 multiply-adds whatever the loop order [1]; the order only changes which memory is read in the innermost loop. In `ijk` the index k walks down a column of B, $8n$ bytes between consecutive accesses, so every access may touch a different cache line and page. Interchanging the two inner loops (`ikj`) keeps $A[i][k]$ in a register and walks along rows of B and C, so every 64-byte line serves 8 consecutive multiply-adds [2], but the whole of B is still read once per row of C. *Cache blocking* splits the loops into $b \times b$ tiles, so a tile of B is reused by all the rows of a tile of A while it is in the cache; the

Prediction	**Statement**	**Reason**	**Checked with**
P1 Loop order	`ikj` is faster than `ijk` at every $n \geq 500$, and its cost per multiply-add has no jump between $n = 1500$ and 2000	B and C are read row by row, with unit stride	Cost per multiply-add
P2 Blocking	The blocked kernel is faster than `ikj` once B no longer fits in the L3 ($n \geq 2000$)	Each tile of B is reused from the cache by the rows of a tile of A	Ratio to `ikj`
P3 Transposed	Faster than `ijk`, but slower than `ikj`	Unit stride, but every element of C is one serial chain of n additions	Ratio to `ikj`
P4 Vectorisation	The AVX2 build speeds up only the loops gcc reports as vectorised (`ikj`, blocked), by at most 4 times	4 doubles per 256-bit instruction	Ratio AVX2 / scalar
P5 Sparse time	The CSR kernel is faster than the scalar dense blocked kernel up to at least 50% density, and the advantage at a given density is larger for band inputs than for uniform ones	The work is about n^3d^2 multiply-adds, plus costs that grow with the nonzeros of C, which fill in faster for uniform inputs	Ratio of medians
P6 Sparse memory	CSR needs less memory than the dense arrays only while the product stays sparse; for uniform inputs with $d \geq 5\%$ it needs more	12 bytes per stored nonzero against 8 per dense entry	Computed storage, peak working set

Table 1: Predictions and how each one is evaluated.

best b depends on the cache sizes and on the interference between tiles [4]. Transposing B once, in $O(n^2)$ operations, instead lets the `ijk` dot product read both operands with unit stride.

Floating-point addition is not associative [3]. All the kernels of this study add the n products of each $C[i][j]$ in increasing k, starting from zero, so they only reorder the visits to different elements and must give results identical bit for bit to `ijk`; the dot product of the transposed kernel therefore remains one serial chain of additions. *Vectorisation* executes one instruction on several values: an AVX2 instruction works on 4 doubles. gcc vectorises loops with `-ftree-vectorize`, which `-O2` enables with the `very-cheap` cost model; `-fvect-cost-model=dynamic` also allows loops that need a run-time check, for example that two pointers do not overlap [7], and `-mavx2` allows the AVX2 instructions [8]. Vectorising the inner loop of `ikj` over j computes 4 elements of C at once without changing the order of the additions of any of them.

2.2 Sparse matrices and the CSR format

The Compressed Sparse Row (CSR) format stores the nonzeros row by row in two arrays, values and column indices, plus $n+1$ row offsets [6]. With 8-byte values, 4-byte indices and 8-byte offsets, a matrix with z nonzeros needs $12z + 8(n+1)$ bytes, against $8n^2$ dense. Gustavson's algorithm computes row i of $C = AB$ as the sum of the rows k of B scaled by the nonzeros $A[i][k]$ [5], accumulated in a dense array of n values. For inputs with density d and uniformly placed nonzeros it performs about n^3d^2 multiply-adds, each with an indirect access. The product fills in: $C[i][j]$ is nonzero with probability $1-(1-d^2)^n$, 18% for $d = 1\%$ and 99% for $d = 5\%$ at $n = 2000$. In a band matrix the nonzeros are confined to $|i-j| \le w$; the product is confined to $|i-j| \le 2w$, so its density is bounded by about twice that of the inputs.

3 Methodology

3.1 Implementations and reused material

The baseline is the validated C implementation of Assignment 1 (https://github.com/Cafabri/Assigment1, commit `0785887`): its kernel, renamed, and its random generator, seeds, timer, batching rule, processor affinity and peak-memory measurement are reused unchanged. C was chosen because it was the fastest implementation of Assignment 1 and its flat row-major array makes the effect of the access order directly observable; Java and Python were not continued, so that the comparisons do not mix a language effect with the optimisation and sparse-storage effects. All kernels are single-threaded and call no numerical library:

- **`ijk`** (baseline) and **`ikj`** (the j and k loops interchanged).
- **blocked**: `ikj` on $b \times b$ tiles, tile loops in the order ii, kk, jj; kk runs outside jj in increasing order, so every element still receives its products in increasing k. Partial edge tiles are handled with bounds.
- **`transposed`**: the `ijk` loop on B^T, prepared before the call; the transposition (`transpose`) is timed separately.
- **`csr`**: Gustavson's algorithm (own code). A symbolic pass counts the nonzeros of every row of C so that C is allocated once with its exact size; a numeric pass accumulates each row in a dense array, with n markers recording which columns the row already holds. The column indices of each row are then sorted, by a scan of the markers when the row has more than $n/16$ nonzeros, otherwise with `qsort`. The products with a zero factor that the dense kernels add are exactly $+0$, so the nonzeros of C are also bit-identical to `ijk`.
- **`convert`**: dense $\to$ CSR conversion of A and B (count, then copy), timed separately.

3.2 Input data

The values are those of Assignment 1: SplitMix64 with seed 42 for A and 43 for B, uniform in $[0, 10)$. The generator is counter-based, so position $p = i \cdot n + j$ has the same value whatever the density. The nonzero positions are: **dense**, all (the matrices of Assignment 1); **uniform**, each position independently when a second SplitMix64 stream (seeds 1042 and 1043) gives a number below d; **band**, positions with $|i - j| \leq w$, w the smallest half-width whose band holds dn^2 positions, each kept with probability dn^2/(band positions) (above 0.95 from $d = 1\%$; `results/patterns.csv`). The sparser matrices are subsets of the denser ones. The sparse method generates its inputs directly in CSR, so large sparse inputs never exist as dense arrays; the program checks that this gives the same arrays as converting the dense matrix.

3.3 Environment

Item	Value
CPU	Intel Core i7-13700KF: 8 P-cores with 2 threads each (logical processors 0–15) and 8 E-cores (16–23)
Cache	48 KB L1 data and 2 MB L2 per P-core, 30 MB shared L3
RAM / OS	31.8 GiB / Windows 11 Pro, power plan *Balanced* (default)
Affinity	Runner and every benchmark process restricted to the P-cores (0–15)
Compiler	gcc 16.1.0 (MSYS2/MinGW64)
Build `scalar`	`-O2 -ffp-contract=off` (the flags of Assignment 1)
Build `avx2`	`-O2 -ffp-contract=off -fvect-cost-model=dynamic -mavx2`
Analysis	CPython 3.11.9, NumPy 2.2 and SciPy 1.15 (reference results), pandas 2.2, matplotlib 3.10

Table 2: Execution environment, recorded in `results/environment.txt`.

The scalar build keeps the flags of Assignment 1: `-O2` without `-O3`, which could interchange the loops by itself [7], and `-ffp-contract=off`, which forbids fused multiply-adds. Both builds use the same source and are compiled with `-fopt-info-vec` (`results/vectorisation_report.txt`). In the scalar build no loop of the kernels is vectorised. In the AVX2 build gcc vectorises the inner loops of `ikj` and blocked with 32-byte vectors, behind a run-time check that B and C do not overlap, and the counting loop of the conversion. The dot products of `ijk` and `transposed` are floating-point sums, which gcc only vectorises when allowed to reorder the additions (`-fassociative-math`) [7]; `transposed` therefore serves as a control without vector code in either build.

3.4 Measurement protocol

A pilot run, not reported, was used to choose the sizes and densities. `run_experiment.py` compiles both builds and runs every configuration in a separate process, never two at the same time, in four experiments:

- **Block size:** blocked, $b \in \{16, 32, 64, 128, 256\}$, $n \in \{1000, 2000\}$, both builds. The block size b^* used afterwards is the one with the lowest median at $n = 2000$ in each build (declared before running).

- **Dense (Part A):** `ijk`, `ikj`, blocked, `transposed` and `transpose` (scalar) and `ikj`, blocked and `transposed` (AVX2), $n \in \{100, 200, 500, 1000, 1500, 2000, 3000, 4000\}$.
- **Density (Part B):** $n = 2000$, $d \in \{0.1, 0.5, 1, 2, 5, 10, 20, 30, 50, 75, 100\}\%$, both distributions: `csr` and `convert` (scalar; the CSR kernel has no vectorised loop), blocked in both builds, and `ijk` at $d \in \{0.1, 10, 100\}\%$.
- **Size (Part B):** `csr` for n from 1000 to 32 000 (doubling) at $d \in \{0.1, 1\}\%$, both distributions, and scalar blocked up to $n = 4000$.

The **budget** is 15 minutes per run, as in Assignment 1 (the process is killed: TIMEOUT), and 16 GiB of matrix storage per process, checked by the program before allocating (for CSR, after the symbolic pass: MEMORY_LIMIT); larger sizes of a failed series are recorded as SKIPPED. The **timer** (`QueryPerformanceCounter`) covers only the kernel call. Dense kernels receive their inputs and a zeroed result prepared outside the timer, as in Assignment 1. The CSR timer includes the symbolic pass and the allocation and sorting of the output, whose size is unknown in advance; its inputs are already in CSR, and the conversion is reported separately and added only where stated. A first call shorter than 10 ms is a separate check call, and the runs are then timed in batches of at least 10 ms. Each process makes 2 warm-up and 5 measured runs; configurations whose first process had a median below 60 s were repeated in 3 independent, interleaved processes (15 runs). Each configuration is summarised by the median and the IQR. Every run writes the checksum of its result, so no call can be removed by the compiler.

3.5 Memory measurement

The *peak working set* of every timing process is measured with `GetProcessMemoryInfo`. The *storage* of each method is computed: $24n^2$ bytes for the dense kernels ($32n^2$ with B^T); for `csr`, $12z + 8(n + 1)$ bytes per matrix (A, B, C) plus $12n$ bytes of work space. Each process holds only the arrays of its own method, so dense and sparse peaks are comparable.

3.6 Correctness

Validation was done in four steps:

1. **Exact cases:** at start-up every kernel checks the identity, zero and 2×2 cases and a 7×7 sparse product with an empty row and partial tiles.
2. **Reference implementation:** for $n = 50$ and 257 (prime, so every block size leaves partial tiles) and five inputs (dense; uniform and band at 5% and 30%), the full result of every kernel, block size and build is compared with NumPy, and the nonzeros of the CSR product with SciPy, on independently regenerated inputs. The tolerance is a relative error of 10^{-11} (the rounding bound nu [3] is 4.4×10^{-13} at $n = 4000$), and reference zeros must be exact.
3. **Bitwise agreement:** every result of step 2 must be identical to scalar `ijk`.
4. **Checksums and nonzeros:** in every configuration the checksum must be the same in every run and agree with NumPy/SciPy, and the nonzeros of A, B and C must equal SciPy's (above 10^8 nonzeros only the agreement between runs is checked).

4 Results

4.1 Correctness

All validation steps passed (`results/correctness.csv`, `results/checksums.csv`). The largest relative error against NumPy was 1.9×10^{-15} (170 comparisons); the 17 kernels checked were bit-identical to scalar `ijk`; the CSR nonzeros equalled SciPy's; and in all 186 configurations the checksum was identical across runs and within 2.6×10^{-13} of the reference (two without reference). The baseline reproduces Assignment 1: `ijk` took 0.467, 10.9 and 73.8 s at $n = 1000$, 2000 and 3000, against 0.47, 10.4 and 73.2 s there.

4.2 Part A: runtime of the dense methods

Block size (Table 3). In the scalar build the differences are small and not monotonic: $b = 32$ is the fastest at both sizes, and the slowest ($b = 64$) needs 1.17–1.19 times its time. In the AVX2 build the time falls as b grows: $b = 16$ needs 1.81–1.85 times the time of $b = 256$, and $b = 128$ and 256 differ by at most 2.4%. The rule gives $b^* = 32$ (scalar) and $b^* = 256$ (AVX2).

	Scalar build		**AVX2 build**	
b	**n = 1000**	**n = 2000**	**n = 1000**	**n = 2000**
16	0.252 (0.9%)	2.02 (0.5%)	0.147 (1.3%)	1.19 (1.3%)
32	0.222 (0.3%)	1.77 (0.7%)	0.101 (2.1%)	0.810 (0.7%)
64	0.261 (3.1%)	2.11 (1.4%)	0.0876 (2.6%)	0.707 (1.0%)
128	0.247 (1.2%)	1.98 (0.8%)	0.0815 (2.3%)	0.661 (1.6%)
256	0.236 (0.7%)	1.92 (1.1%)	0.0815 (2.3%)	0.646 (0.8%)

Table 3: Median time of the blocked kernel in seconds, with the IQR as a percentage of the median in parentheses (15 runs in 3 processes each).

n	**ijk (s)**	**ikj (s)**	**blocked (s)**	**transposed (s)**	**transpose (s)**
100	3.01×10^{-4} (5.2%)	2.48×10^{-4} (1.9%)	2.26×10^{-4} (2.3%)	2.57×10^{-4} (4.5%)	3.85×10^{-6} (7.9%)
200	2.72×10^{-3} (1.2%)	1.80×10^{-3} (3.0%)	1.76×10^{-3} (2.3%)	2.84×10^{-3} (3.0%)	1.07×10^{-5} (3.5%)
500	0.0468 (1.6%)	0.0285 (1.0%)	0.0274 (2.1%)	0.0459 (2.0%)	2.13×10^{-4} (9.0%)
1000	0.467 (1.3%)	0.232 (1.0%)	0.219 (2.1%)	0.378 (1.6%)	9.00×10^{-4} (3.3%)
1500	1.77 (9.4%)	0.858 (1.4%)	0.741 (1.0%)	1.35 (2.2%)	2.40×10^{-3} (20.6%)
2000	10.9 (4.7%)	2.84 (2.7%)	1.77 (1.1%)	3.76 (3.4%)	6.40×10^{-3} (33.4%)
3000	73.8 (0.2%)	10.7 (1.4%)	5.97 (0.8%)	13.3 (0.5%)	0.0208 (1.3%)
4000	234 (1.2%)	27.0 (2.8%)	14.2 (1.6%)	32.4 (1.7%)	0.0477 (9.8%)

Table 4: Scalar build: median time per multiplication in seconds (IQR as a percentage of the median). *transpose* is the transposition of B alone, not included in *transposed*. 15 runs in 3 processes, except `ijk` at $n = 3000$ and 4000 (5 runs in one process). For $n \leq 200$ each run is a batch of calls lasting at least 10 ms.

Loop order and blocking (Table 4, Figure 1). `ikj` is faster than `ijk` at every size, by 1.6 at $n = 500$, 2.0–2.1 at $n = 1000$–1500, 3.8 at $n = 2000$ and 8.6 at $n = 4000$. Its cost per multiply-add is 0.23 ns up to $n = 1000$ and rises to 0.25, 0.36 and 0.42 ns at $n = 1500$, 2000 and 4000; that of `ijk` rises from 0.52 to 1.36 ns between $n = 1500$ and 2000 and

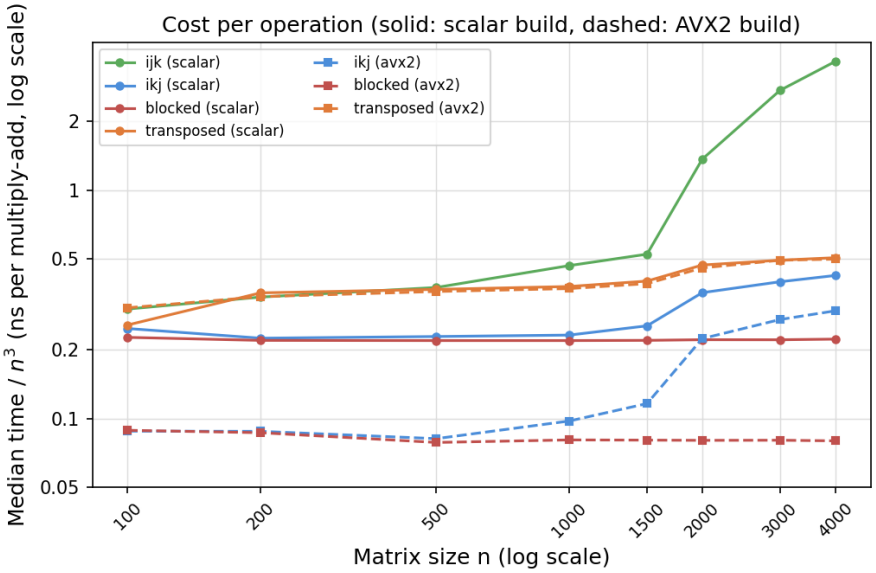


Figure 1: Median time divided by n^3 (ns per multiply-add), both builds. A runtime exactly proportional to n^3 would give a horizontal line.

reaches 3.65 ns at $n = 4000$. The blocked kernel keeps 0.219–0.222 ns from $n = 500$ to 4000 (fitted exponent 3.01): it is 1.02–1.16 times faster than `ikj` up to $n = 1500$, 1.61–1.90 times faster for $n \geq 2000$, and 16.4 times faster than `ijk` at $n = 4000$.

Transposed operand. `transposed` takes 0.96–1.02 times the time of `ijk` at $n = 200$ and 500 (0.36–0.37 ns in both) and is faster from $n = 1000$ (1.23 times) to 4000 (7.2 times). It is slower than `ikj` at every size, by 1.6 times for $200 \leq n \leq 1500$ and 1.2 at $n = 4000$. The transposition takes at most 0.5% of the kernel time for $n \geq 200$.

Vectorisation. The AVX2 build makes the blocked kernel 2.54–2.79 times faster at every size (0.080 ns per multiply-add from $n = 500$). For `ikj` the gain falls with n: 2.2–2.8 up to $n = 1500$, 1.59 at $n = 2000$ and 1.43 at $n = 4000$, where the vectorised `ikj` needs 3.71 times the time of the vectorised blocked kernel. `transposed` takes 0.96–1.00 times its scalar time from $n = 200$ (16% slower at $n = 100$). The vectorised blocked kernel is 3.4–6.5 times faster than `ijk` up to $n = 1500$ and 17.0, 34.0 and 45.8 times faster at $n = 2000$, 3000 and 4000.

Variability. The IQR is at most 3% of the median in 78.5% of the 214 configurations. Among medians above 1 ms the largest values are those of the transposition alone (up to 33.4%, for calls of a few ms), `ijk` at $n = 1500$ (9.4%) and at $n = 2000$ in the density experiment (up to 13.5%, process medians up to 22% apart), and the vectorised `ikj` at $n = 1500$ (10.8%).

4.3 Part A: memory

The peak working set is identical for `ijk`, `ikj` and blocked (95.6 MiB at $n = 2000$, 370 MiB at $n = 4000$) and grows by 24.0 bytes per element for $n \geq 1000$, the three float64 matrices, plus about 4 MiB of fixed program memory. `transposed` grows by 32.0 bytes per element (492 MiB at $n = 4000$), because it also holds B^T. The AVX2 build does not change the

memory. None of the optimisations needs more memory than the baseline except the transposed operand.

4.4 Part B: runtime vs density

	Density (%)				Dense blocked (s)		Scalar blocked /	
d (%)	**A**	**C**	**CSR (s)**	**Conv. (s)**	**scalar**	**AVX2**	**CSR**	**CSR + conv.**
Uniform nonzeros								
0.1	0.1	0.2	1.73×10^{-4}	7.84×10^{-3}	1.76	0.640	10100	219
0.5	0.49	4.81	5.31×10^{-3}	8.76×10^{-3}	1.77	0.646	333	126
1	0.99	18	9.52×10^{-3}	8.98×10^{-3}	1.78	0.645	187	96
2	1.99	55	0.0295	1.00×10^{-2}	1.77	0.642	59.9	44.7
5	4.99	99.3	0.0641	0.0139	1.76	0.643	27.5	22.6
10	9.98	100	0.106	0.0207	1.79	0.647	16.9	14.1
20	20	100	0.268	0.0333	1.76	0.643	6.58	5.85
30	30	100	0.536	0.0420	1.76	0.641	3.29	3.05
50	50	100	1.43	0.0529	1.76	0.639	1.23	1.18
75	75	100	3.38	0.0379	1.76	0.648	0.522	0.516
100	100	100	6.04	0.0175	1.76	0.642	0.292	0.291
Band nonzeros								
0.1	0.1	0.15	9.32×10^{-5}	7.90×10^{-3}	1.76	0.642	18900	220
0.5	0.5	1.03	7.66×10^{-4}	7.97×10^{-3}	1.77	0.642	2310	202
1	1	2.03	1.76×10^{-3}	8.03×10^{-3}	1.76	0.638	1000	180
2	2	4.01	5.11×10^{-3}	8.37×10^{-3}	1.76	0.641	346	131
5	5	9.98	0.0172	8.57×10^{-3}	1.77	0.641	102	68.4
10	10	19.6	0.0572	9.26×10^{-3}	1.76	0.640	30.7	26.4
20	20	37.8	0.211	9.89×10^{-3}	1.76	0.645	8.36	7.98
30	30	54.7	0.465	0.0109	1.77	0.644	3.8	3.72
50	50	82.9	1.30	0.0128	1.77	0.645	1.37	1.35
75	75	100	3.27	0.0153	1.76	0.643	0.54	0.537
100	100	100	6.06	0.0175	1.77	0.640	0.291	0.291

Table 5: $n = 2000$. Requested density d, measured density of A and of C, median times of the CSR kernel, of the conversion of A and B and of the dense blocked kernel, and ratio of the scalar blocked time to the CSR time without and with the conversion (above 1: CSR is faster).

Dense methods and fill-in. The dense kernels do not depend on the zeros: across the 22 inputs the blocked median varies by 1.6% in each build, and `ijk` took 9.8–11.5 s, against 10.9 s on the dense matrices. For uniform inputs the density of C follows the model $1-(1-d^2)^n$ (Figure 3): 18% at $d = 1\%$, 55% at 2% and 99.3% at 5%. For band inputs it is 1.5–2.1 times the input density up to $d = 20\%$, and reaches 100% only at $d = 75\%$.

Time (Table 5, Figure 2). CSR is faster than the scalar blocked kernel up to $d = 50\%$ in both distributions: by 10 100 (uniform) and 18 900 (band) times at $d = 0.1\%$, 187 and 1000 times at 1%, 16.9 and 30.7 at 10%, and 1.23 and 1.37 at 50%; at 75% and 100% it needs 1.9 and 3.4 times the dense time. The advantage is larger for band inputs at every density from 0.1% to 50%. Interpolating the ratio in log scale, the crossover is at 55.1% (uniform) and 57.3% (band) against the scalar blocked kernel, and at 32.9% and 35.3% against the vectorised one. Divided by n^3d^2, the CSR time is 0.65–0.76 ns for $d \geq 20\%$ and grows at low density, up to 21.7 (uniform) and 11.6 ns (band) at 0.1%.

Conversion. Converting A and B takes 7.8–53 ms: 45 (uniform) and 85 (band) times the CSR kernel at $d = 0.1\%$, and less than 20% of it from $d = 10\%$. Including it moves

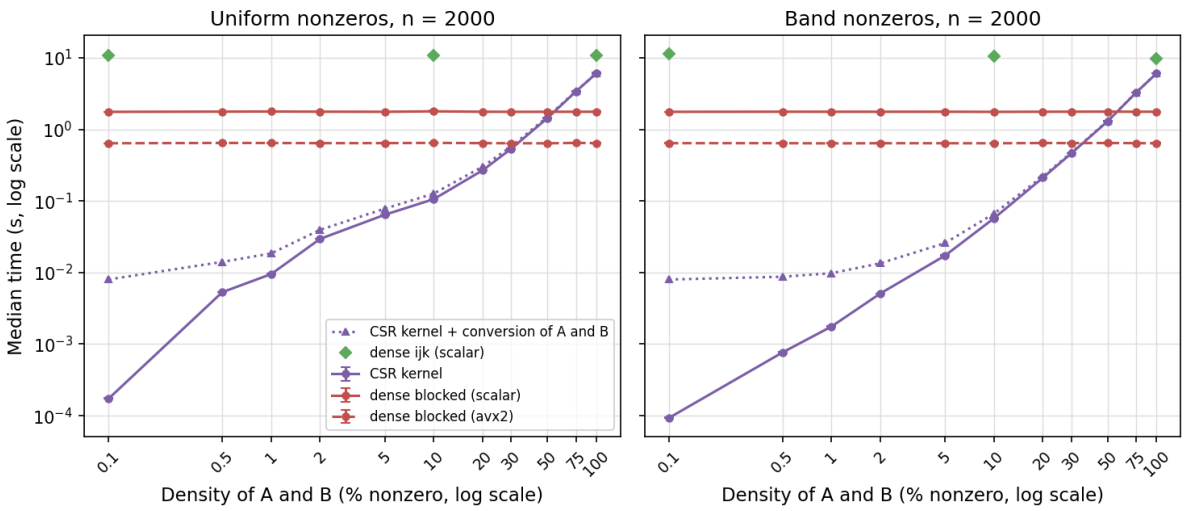


Figure 2: Median time vs density of A and B at $n = 2000$ (log-log scale). Left: uniform nonzeros; right: band nonzeros. `ijk` was run at three densities only.

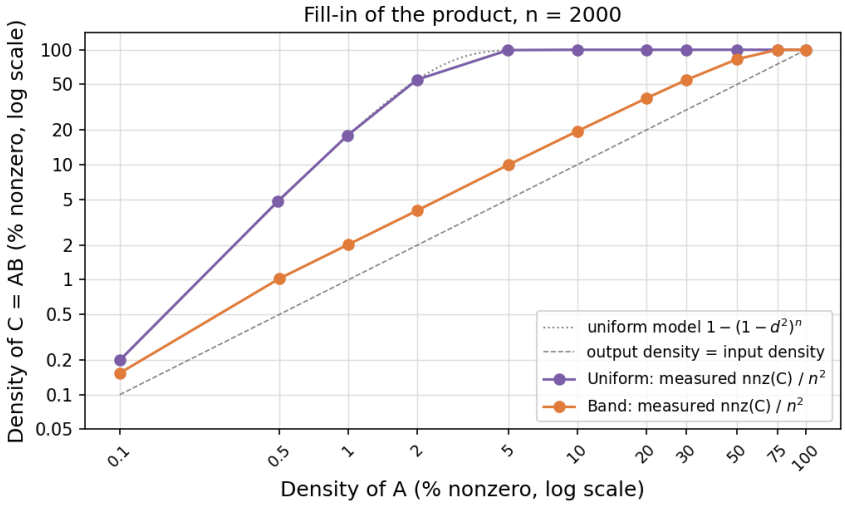


Figure 3: Density of $C = AB$ vs density of the inputs at $n = 2000$ (log-log scale). The dotted line is the uniform model $1 - (1 - d^2)^n$.

the crossovers by at most 1.2 percentage points and leaves CSR about 220 times faster than the scalar dense kernel at $d = 0.1\%$. The uniform conversion is slowest at $d = 50\%$ (52.9 ms, against 17.5 ms at 100%); the band conversion grows steadily with d.

4.5 Part B: memory

At $n = 2000$ (Figure 4), the CSR storage is smaller than the $24n^2$ bytes of the dense matrices up to 49.9% density (uniform) and 56.1% (band), and 1.5 times larger at 100%; at $d = 5\%$, with an almost full product, the uniform CSR matrices still need 0.55 times the dense storage. The peak working set of CSR goes from 4.3 MiB at $d = 0.1\%$ to 141.5 MiB at 100%, against 95.6 MiB for the dense kernel at every density.

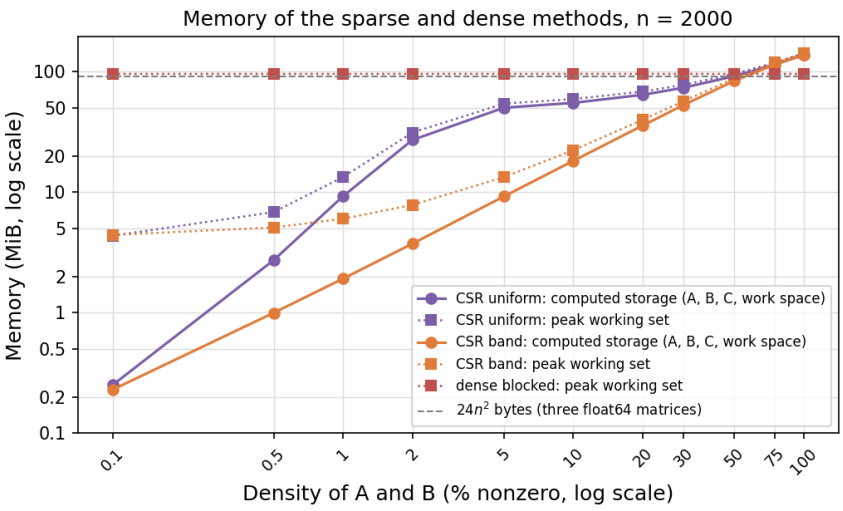


Figure 4: Memory vs density at $n = 2000$ (log-log scale): computed CSR storage (A, B, C and work space) and peak working set of the CSR and dense blocked processes. The dashed line is $24n^2$ bytes.

4.6 Part B: runtime vs matrix size

The fitted exponents of the CSR time from $n = 1000$ to 32 000 (Table 6, Figure 5) are 3.33 (uniform 0.1%), 2.74 (uniform 1%), 2.46 (band 0.1%) and 2.52 (band 1%). The uniform products fill in as n grows (from 9.4% to 95.9% at $d = 1\%$), while the band products keep a constant density. On the inputs where both ran ($n \leq 4000$), CSR is 158–24 300 times faster than the scalar blocked kernel. The CSR storage grows with the nonzeros of C: at $n = 32\,000$ it is 48–473 MiB for the band inputs and 394 MiB and 11 477 MiB for uniform 0.1% and 1%.

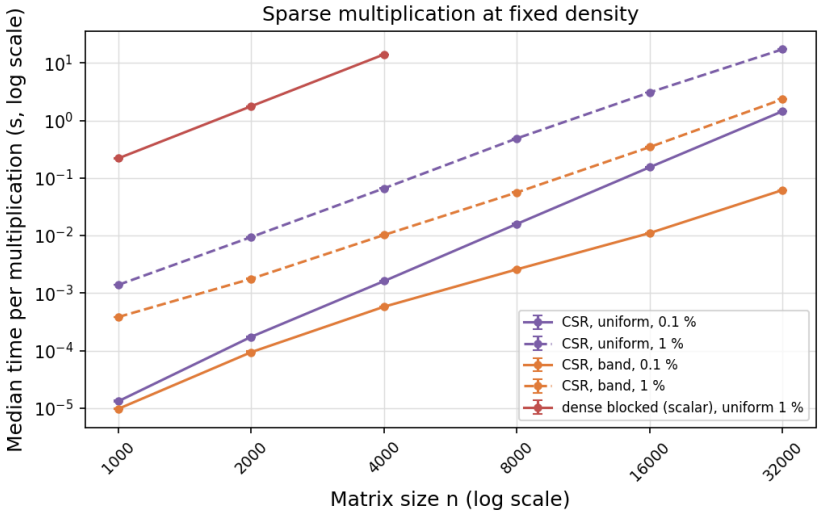


Figure 5: Median CSR time at fixed density vs n (log-log scale), and the scalar dense blocked kernel up to $n = 4000$.

Distr.	d (%)	n	CSR (s)	C (%)	Storage	Peak	Blocked (s)
Uniform	0.1	1000	1.32×10^{-5} (4.7%)	0.098	0	4	0.219
		2000	1.73×10^{-4} (3.5%)	0.2	0	4	1.77
		4000	1.61×10^{-3} (1.4%)	0.4	1	5	14.1
		8000	0.0159 (1.2%)	0.8	8	12	—
		16000	0.154 (0.7%)	1.59	53	57	—
		32000	1.45 (0.4%)	3.15	394	398	—
	1	1000	1.39×10^{-3} (1.9%)	9.37	1	5	0.220
		2000	9.40×10^{-3} (1.5%)	18	9	13	1.77
		4000	0.0662 (1.6%)	32.9	64	68	14.1
		8000	0.488 (0.5%)	55	418	422	—
		16000	3.08 (0.3%)	79.8	2397	2401	—
		32000	17.3 (0.4%)	95.9	11477	11481	—
Band	0.1	1000	9.72×10^{-6} (3.8%)	0.1	0	4	0.219
		2000	9.38×10^{-5} (8.3%)	0.15	0	4	1.76
		4000	5.79×10^{-4} (1.6%)	0.2	1	5	14.1
		8000	2.58×10^{-3} (2.6%)	0.21	3	7	—
		16000	0.0110 (2.3%)	0.2	12	17	—
		32000	0.0617 (1.7%)	0.2	48	53	—
	1	1000	3.83×10^{-4} (1.9%)	2.05	0	5	0.220
		2000	1.77×10^{-3} (2.5%)	2.03	2	6	1.77
		4000	0.0102 (2.3%)	2.01	7	12	14.1
		8000	0.0561 (0.9%)	2	30	34	—
		16000	0.345 (1.0%)	2	118	122	—
		32000	2.35 (0.6%)	1.99	469	473	—

Table 6: CSR at fixed density: median time (IQR), density of the product, computed storage and peak working set (MiB), and the scalar blocked kernel on the same inputs.

4.7 Largest feasible size

No process exceeded the time or memory budget (`results/run_status.csv`), so every declared size finished. Table 7 shows the largest size of each method.

Method	Largest feasible n	Longest median	Largest peak
`ijk` (scalar)	4000	234 s	370 MiB
`ikj` / blocked (scalar)	4000	27.0 s / 14.2 s	370 MiB
blocked (AVX2)	4000	5.1 s	370 MiB
`transposed` (scalar)	4000	32.4 s	492 MiB
CSR, uniform 1%	32 000	17.3 s	11 481 MiB
CSR, band 1%	32 000	2.35 s	473 MiB

Table 7: Limits under the budget of 15 minutes per run and 16 GiB of matrix storage. The largest sizes attempted were $n = 4000$ (dense) and 32 000 (CSR); none exceeded the budget.

5 Discussion

5.1 Evaluation of the hypotheses

In answer to question (A): reading memory with unit stride and reusing tiles from the cache remove most of the slowdown of `ijk` at large n without extra memory, and vectorisation multiplies the gain of the kernels gcc can vectorise. In answer to question (B): at $n = 2000$, CSR is faster than the dense kernels up to a density of about one half (scalar) or one third (vectorised), and needs less storage up to about one half, for both distributions. Table 8 gives the verdict for each prediction.

Prediction	**Observed**	**Verdict**
P1 Loop order	`ikj` faster at every $n \geq 500$ (1.6–8.6 times), but its cost per multiply-add still rises from 0.25 to 0.36 ns between $n = 1500$ and 2000	Partly confirmed (smaller jump, not none)
P2 Blocking	1.61–1.90 times faster than `ikj` for $n \geq 2000$; constant 0.22 ns per multiply-add	Confirmed
P3 Transposed	Slower than `ikj` at every size; faster than `ijk` only from $n = 1000$, equal within 4% for $200 \leq n \leq 500$	Partly confirmed (not faster below $n = 1000$)
P4 Vectorisation	Blocked 2.5–2.8 times, `ikj` 1.4–2.8 times, the non-vectorised `transposed` within 4% from $n = 200$	Confirmed
P5 Sparse time	CSR faster than scalar blocked up to 50% (crossovers 55% and 57%); larger advantage for band inputs at every density up to 50%	Confirmed
P6 Sparse memory	CSR storage smaller than dense up to 50% (uniform) and 56% (band), including uniform inputs with $5\% \leq d \leq 50\%$	Refuted (threshold wrong)

Table 8: Verdict for each prediction of Table 1.

5.2 Effect of the dense optimisations

The dense kernels perform the same n^3 multiply-adds in the same order for each element, so their differences come only from the order in which memory is read and from the instructions used. None of the explanations below was measured with hardware counters.

Memory access (`ikj`).

- Up to $n = 1500$, B (at most 18 MB) fits in the 30 MB L3 and `ikj` is 1.6–2.1 times faster than `ijk`. Beyond it, the cost per operation of `ijk` grows 2.6 times and that of `ikj` only by 40%.
- `ikj` uses every page and cache line of B completely, so its smaller jump is consistent with the L3-capacity explanation of Assignment 1 (B, 32 MB at $n = 2000$, is read again from memory for every row of C); the much larger jump of `ijk` suggests that its stride adds to it. The share of cache and TLB remains a hypothesis.

- At $n = 2000$ `ikj` reads one 8-byte element of B per multiply-add, equivalent to about 22 (scalar) and 36 GB/s (AVX2), plausible main-memory rates; no bandwidth benchmark was run.

Cache blocking.

- The blocked kernel removes the dependence on n, as expected if each tile of B is reused from the cache instead of being read again from memory [4].
- The best block sizes are consistent with the cache sizes: with $b = 32$ the three tiles occupy $3 \times 8b^2 = 24$ KB (48 KB L1), and with $b = 256$ 1.5 MB (2 MB L2).
- In the AVX2 build small tiles are 1.8 times slower, consistent with an inner loop of b elements too short for the vector loop (4 doubles, unrolled 4 times) and its aliasing check.

Transposed operand.

- It gives `ijk` unit stride but keeps one serial chain of n additions per element. Up to $n = 500$, where `ijk` still finds B in the cache, both cost 0.36–0.37 ns per multiply-add, while `ikj`, whose consecutive additions go to different elements, costs 0.23 ns: consistent with the dot products being limited by the latency of the additions [2].
- The transposition costs at most 0.5% of the kernel, so it is a valid fix for the memory problem of `ijk`, but not as good as reordering the loops.

Vectorisation.

- The loop order is what allows it: `ikj` updates independent elements and is vectorised; the dot product cannot be without changing the rounding, and `transposed` gains nothing.
- The gain never reaches the 4 doubles of a register (at most 2.8), and in `ikj` it falls to 1.4 at large n: with faster arithmetic, the kernel that reads B from memory for every row waits longer for memory, which blocking avoids.
- The optimisations combine: over `ijk` at $n = 4000$, blocking alone gives 16.4 times, vectorisation with the loop reordering it needs 12.4 times, and both 45.8 times.
- The 16% slowdown of `transposed` at $n = 100$ in the AVX2 build is not explained by the measurements; code alignment is one possible cause.

5.3 Effect of sparse storage

Time and crossover.

- CSR performs only about n^3d^2 of the n^3 multiply-adds but pays more for each: for $d \geq 20\%$ its time divided by n^3d^2 (0.65–0.76 ns) is 3.0–3.5 times the 0.22 ns of the scalar blocked kernel and 8–9.5 times the 0.080 ns of the vectorised one, because each product reads a column index, updates the accumulator at a scattered position and checks a marker.

- If both costs were constant, the crossover would be where d^2 equals their ratio: $\sqrt{0.22/0.72} \approx 55\%$ (scalar) and $\sqrt{0.080/0.72} \approx 33\%$ (vectorised), close to the measured 55% and 33% (uniform). The crossover therefore depends on how optimised the dense alternative is.

Low density and distribution.

- At low density the cost per product grows, because the products fall as d^2 while the costs per row of C (marker initialisation, sorting) and per nonzero of C do not.
- This is why the distribution matters: at $d = 1\%$ both perform about the same number of products, but the uniform product has 18% nonzeros and the band product 2%, and the band kernel is 5.4 times faster. The adjacent rows of B selected by a band row may also help; this was not separated from the fill-in.

Conversion.

- Negligible at medium and high densities. At $d = 0.1\%$ it is bounded below by reading the two dense matrices twice (7.8 ms against 0.17 ms for the kernel), and CSR still remains about 220 times faster.
- Its peak at $d = 50\%$ for uniform inputs is consistent with the branch on `v != 0` being unpredictable when half of the random entries are zero, unlike band inputs, whose nonzeros form almost contiguous runs; this was not measured.

Memory. P6 compared the 12 bytes of a CSR nonzero with the 8 of a dense entry for the product only. The relevant comparison is with all three dense matrices: even when C is full and its CSR form needs $12n^2$ bytes, the sparse A and B save more than that up to about $d = 50\%$.

Scaling with n.

- Band inputs (exponent 2.5): fixed per-row costs dominate at small n and are amortised; the time per product of band 1% falls from 3.8 to 0.71 ns between $n = 1000$ and 32 000.
- Uniform 0.1% (exponent 3.33): the rows of C have about d^2n^2 nonzeros (about 1000 at $n = 32\,000$), sorted with `qsort`, so the cost per product grows with $\log n$; not confirmed with a profile.
- Limit of sparse storage: at $n = 32\,000$ the uniform 1% product has 96% nonzeros and its CSR storage (11.2 GiB) exceeds the 7.6 GiB of a dense C, while the band product needed 473 MiB. Three dense matrices would need 22.9 GiB, above the budget, and the cubic extrapolation of the blocked kernel gives about 2 hours per run; neither was run.

5.4 Observed bottlenecks

Inferred from the timings, without hardware counters:

- **`ijk`:** the strided reads of B once B no longer fits in the L3 (up to 3.65 ns per multiply-add at $n = 4000$).

- **`ikj` (both builds):** reading the whole of B from main memory for every row of C for $n \geq 2000$; the faster the arithmetic, the larger its share.
- **`transposed`, and `ijk` while B fits in the cache:** the latency of one serial chain of additions per element.
- **Blocked kernel:** neither memory nor n; its constant cost is set by the arithmetic, which only vectorisation reduced (0.22 to 0.080 ns).
- **CSR kernel:** the indirect access of every product; at low density, the per-row costs and the sorting of C; with uniform inputs, the fill-in of C, which also sets its memory (11.2 GiB at $n = 32\,000$, the closest to the budget).
- **Conversion:** reading the dense arrays, which dominates only below about 1% density.

5.5 Limitations

- One computer, compiler and set of flags; other caches would move the jumps, the best block size and the crossovers. The power plan was *Balanced*, not a fixed frequency.
- Only five square block sizes were tried, without register blocking, so the blocked kernel is not the fastest possible dense kernel; the crossovers are interpolated between 11 densities.
- The cache, TLB, latency and branch-prediction explanations are inferred from timings only, without hardware counters or profiles.
- Part B used $n = 2000$ and two distributions; the crossovers depend on n through the fill-in, and other patterns may behave differently. The CSR code was not compared with an optimised library, and its sorting rule was not tuned.
- `ijk` at $n \geq 3000$ ran in one process, and `ijk` at $n = 2000$ showed process medians up to 22% apart. No configuration reached the budget, so the actual largest feasible sizes were not determined.

6 Conclusions

For the programs and the conditions described in Section 3:

- Changing only the order of memory accesses makes the dense multiplication much faster with the same operations and memory: `ikj` is 1.6–8.6 times faster than `ijk` for $500 \leq n \leq 4000$ and the blocked kernel 1.7–16.4 times, with a constant 0.22 ns per multiply-add and without the jump of Assignment 1.
- Transposing B removes most of the slowdown of `ijk` at large n, at a negligible cost and one more matrix of memory, but stays 1.2–1.6 times slower than `ikj`.
- Vectorisation multiplies the effect of the optimisations it applies to: 2.5–2.8 times for the blocked kernel, 45.8 times faster than `ijk` at $n = 4000$, and nothing for the dot-product kernel.

- At $n = 2000$, CSR is faster than the dense blocked kernel up to about 55–57% density (scalar) and 33–35% (vectorised), and needs less storage up to 50–56%. Its advantage is larger when the product stays sparse (band) than when it fills in (uniform), and the conversion only matters at very low densities.

The optimisations make each operation cheaper; sparse storage avoids operations at a higher cost per operation, so its advantage depends on the density, on the distribution of the nonzeros (through the fill-in of the product) and on how fast the dense alternative is, not on the percentage of zeros alone. Open questions are the share of the cache and of address translation in the slowdown of the dense kernels, which hardware counters could answer, and how the crossovers move with n and with several threads.

Reproducibility materials

The repository linked on the first page contains the source code, the build and run commands (`README.md`) and, in `results`, the raw measurements of every run, the status and peak memory of every process, the patterns of the sparse inputs, the block size chosen, the vectorisation report, the environment record and the correctness results. The study is reproduced on Windows with `run_experiment.py` (3.3 hours of benchmark processes on this machine), `analysis/check_correctness.py`, `analysis/plot_results.py` (figures and tables) and `analysis/report_numbers.py` (every other number in the text).

Use of generative AI

Claude Opus 5.5 (Anthropic), used through Claude Code, was used during this assignment to help write and debug the benchmark, run and analysis code, and to draft and revise the text of the report from the measured results. All the measurements come from running the code on the machine described above. The author defined the study, reviewed the code and the AI-generated text, checked the numbers against the raw data and the references, and is responsible for the conclusions.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[2] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2019.

[3] D. Goldberg, "What every computer scientist should know about floating-point arithmetic," *ACM Comput. Surv.*, vol. 23, no. 1, pp. 5–48, Mar. 1991, doi: 10.1145/103162.103163.

[4] M. D. Lam, E. E. Rothberg, and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," in *Proc. 4th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS IV)*, 1991, pp. 63–74, doi: 10.1145/106972.106981.

[5] F. G. Gustavson, "Two fast algorithms for sparse matrices: multiplication and permuted transposition," *ACM Trans. Math. Softw.*, vol. 4, no. 3, pp. 250–269, 1978, doi: 10.1145/355791.355796.

[6] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Philadelphia, PA, USA: SIAM, 2003.

[7] Free Software Foundation, "Options that control optimization," in *Using the GNU Compiler Collection (GCC)*. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html.

[8] Free Software Foundation, "x86 options," in *Using the GNU Compiler Collection (GCC)*. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/x86-Options.html.